

Extra Edition (Input for a Z80-based 8-bit Compiler)

 flexonsbd.blogspot.com/2023/07/z80.html

Wednesday, July 5, 2023

Attempts to Create a Compiler, Part 3: Extra Edition (Input for a Z80-based 8-bit Compiler)

I tried inputting the Z80 version of the 8-bit compiler, stellar.



In the May 20th post, "Attempting to Create a Compiler Part 1: Inputting the List of 'High-Class C Language'", I mentioned " Techniques for Creating New Languages Compatible with Z80CPU (by Hiroiyuki Onuki, MIA Co.) " as a reference, and I tried inputting the assembly listing for stellar, a compiler language that runs on CP/M-80, which is included in that book .

Although it was over 120 pages long, I managed to input it all and assemble it using Macro-80 on CP/M-80 to create an executable file.

The file you created

- Generates stellar.com compiler object files
- Convert the object files generated by convobj.com to HEX format

The two steps are as follows: the compilation procedure is shown as an example below, but it involves compiling at stellar.com, converting to HEX format at convobj.com, saving to disk at save.com, and then running it.

```

1      d>stellar mdump
2      Stellar compiler Rev 1.01 ( CP/M-80,MSX-DOS Version )
3      Copyright (c) 1984 H.Ohnuki / MIA
4      Program  name : mdump
5      Function name : getchr
6      Function name : putchar
7      Function name : putnl
8      Function name : putstr
9      Function name : puthex
10     Function name : puthex1
11     Function name : bdos
12     Program  04EF (0100-05EE)
13     Data      0047 (4000-4046)
14     ** End of compile, No error(s)
15     d>convobj mdump
16     Stellar utility, convert object ==> intel HEX
17     Rev 1.00 Copyright (c) 1984 H.Ohnuki / MIA
18     Program  name : mdump = 0100
19     Function name : getchr = 0482
20     Function name : putchar = 04A1
twenty one  Function name : putnl = 04C4
twenty two  Function name : putstr = 0511
twenty      Function name : puthex = 0549
three       Function name : puthex1 = 057A
twenty four Function name : bdos = 05B6
twenty five Constant name : _work = 4000
26          Constant name : _var = 4030
27          Constant name : _code = 0100
28          Constant name : ngetchr = 0001
29          Constant name : nputchr = 0002
30          Variable name : d = 4030
31          Variable name : i = 4031
32          Variable name : sttadr = 4032
33          Variable name : endadr = 4036
34          Variable name : sadr = 403A
35          Variable name : eadr = 403C
36          Variable name : ch = 403E
37          Data      name : msttadr = 020D

```

```

38      Data      name : mendadr = 0217
39      Data      name : header = 0221
40      End address   : 05EE
41      Program size  : 04EF  [ 5 Page ]
42      d>save 5 mdump.com
43      d>
44      d>mdump
45      sttadr= $1234
46      endadr= $1567
47      +0 +1 +2 +3 +4 +5 +6 +7 +8 +9 +A +B +C +D +E +F  0123456789ABCDEF
48      1230: 55 E5 22 DD 55 CD E8 12 3E 32 2B CD 79 23 21 00
49      U." .U...>2+.y#!.
50      1240: 00 22 DF 55 3A 93 55 FE 7B 28 1F FE 9A C2 19 2A  ." .U: .U.
51      {( .....*
52      1250: CD 1A 25 CD C5 17 FE 7B C2 19 2A 2A 6D 55 22 DF  ..%....
53      {..**mU".
54      1260: 55 CD E8 12 3E 32 2B CD 79 23 CD 50 15 2A DD 55
55      U...>2+.y#.P.*.U
56      1270: CD 48 22 2A DB 55 CD 48 22 21 F9 12 CD A5 22 2A
57      .H"* .U.H"!...."*
58      1280: 69 55 2B 2B 22 D9 55 CD 93 11 2A DF 55 7C B5 20
59      iU++" .U...*.U|.
60      1290: 19 2A DB 55 3E 21 CD 79 23 3E 34 CD 5E 23 2A D7
61      .* .U>! .y#>4.^#*.
62      12A0: 55 3E 20 CD 35 15 3E C2 18 19 CD 48 22 2A DB 55  U>
63      .5.>....H"* .U
64      12B0: CD 48 22 21 04 13 CD A5 22 2A D7 55 3E 30 CD 35
65      .H"!...."* .U>0.5
66      12C0: 15 3E D2 DC 79 23 CD 68 15 E1 22 6D 55 01 06 00
67      .>..y#.h.."mU...
68      12D0: 21 DB 55 CD 0E 25 01 0F 00 21 73 55 CD 0E 25 21
69      !.U..%...!sU..%!
70      12E0: 73 55 CD 39 08 C3 1A 25 23 22 6D 55 EB 2A 6B 55
71      sU.9...%#"mU.*kU
72      12F0: B7 ED 52 EB D0 22 6B 55 C9 0A 3A B9 B9 21 BB BB
73      ..R.."kU.....!..
74      +0 +1 +2 +3 +4 +5 +6 +7 +8 +9 +A +B +C +D +E +F  0123456789ABCDEF
75      1300: 96 DA 00 00 08 3A B9 B9 21 BB BB 86 77 FE 23 CA
76      .....!...w.#.

```

```

77      1310: 97 13 3D C2 19 2A 21 73 55 CD C0 08 B7 C4 7F 28
78      ..=..*!sU.....(
79      1320: 21 73 55 7E E6 2F FE 02 C2 19 2A 01 0F 00 CD FE
80      !sU~/....*.....
81      1330: 24 21 73 55 CB EE CD 39 08 2A DB 55 E5 2A 80 55
82      $!sU...9.*.U.*.U
83      1340: 22 DB 55 CD 1A 25 FE 2C C2 19 2A CD 1A 25 CD C5
84      ".U..%.,...*..%..
85      1350: 17 FE 7B C2 19 2A 3E 32 2A DB 55 CD 79 23 CD 50  ..
86      {..*>2*.U.y#.P
87      1360: 15 CD 93 11 3E 21 2A DB 55 CD 79 23 3E 35 CD 5E
88      ....>!*.U.y#>5.^
89      1370: 23 2A D7 55 3E 20 CD 35 15 3E C2 DC 79 23 CD 68  #*.U>
90      .5.>..y#.h
91      1380: 15 E1 22 DB 55 01 0F 00 21 73 55 CD 0E 25 21 73
92      ..".U...!sU..%!s
93      1390: 55 CD 39 08 C3 1A 25 21 96 55 7E B7 C2 19 2A 2F
94      U.9...%!..U~...*/
95      13A0: 77 CD 1A 25 FE 2C C2 19 2A CD 1A 25 CD 00 18 47
96      w..%.,...*..%...G
97      13B0: 3A 98 55 3D C2 48 2A 78 FE 7B C2 19 2A FD 7E FD  :.U=.H*x.
98      {..*..~.
99      13C0: B7 28 11 21 F7 17 3D 28 08 21 FF 13 CD 9A 22 18  .(!!..=
100     (!!....".
101     13D0: 08 CD 9A 22 3E 47 CD 5E 23 CD 50 15 CD 93 11 2A
102     ...">G.^#.P....*
103     13E0: D7 55 3E 10 CD 35 15 30 0C E5 3E 05 CD 5E 23 E1
104     .U>..5.0..>..^#.
105     13F0: 3E C2 CD 79 23 AF 32 96 55 CD 68 15 C3 1A 25 02
106     >..y#.2.U.h...%.
107     +0 +1 +2 +3 +4 +5 +6 +7 +8 +9 +A +B +C +D +E +F  0123456789ABCDEF
108     1400: 06 B8 2A E3 55 E5 2A E1 55 E5 21 00 00 22 E3 55
109     ..*.U.*.U.!..".U
110     1410: 22 E1 55 CD C5 17 FE 94 C2 19 2A 3E B7 CD 5E 23
111     ".U.....*>..^#
112     1420: 3E CA 21 00 00 CD 79 23 2A 69 55 2B 2B 22 E1 55
113     >.!...y#*iU++".U
114     1430: CD 1A 25 CD 91 10 3A 93 55 FE BD 28 2A FE 95 28
115     ..%....:U..(*..(

```

```

116      1440: 08 2A E1 55 CD C2 23 18 0D CD 1A 25 FE 93 28 17
117      .*.U..#....%..(.
118      1450: CD 6F 14 CD 91 10 2A E3 55 7C B5 C4 C2 23 E1 22
      .o....*.U|...#."
      1460: E1 55 E1 22 E3 55 C9 CD 6F 14 CD 1A 25 18 A4 2A
      .U..".U..o....%..*
      1470: E3 55 3E C3 CD 79 23 2A E1 55 CD C2 23 2A 69 55
      .U>..y#*.U..#*iU
      1480: 2B 2B 22 E3 55 C9 FE 3B C2 19 2A 3E C3 2A D9 55
      ++".U...;...*>.*.U
      1490: CD 79 23 2A 69 55 2B 2B 22 D9 55 C3 1A 25 FE 99
      .y#*iU++".U..%..
      14A0: C2 19 2A CD 1A 25 3D C2 19 2A 21 73 55 CD C0 08
      ..*...%=...*!sU...
      14B0: B7 28 0D 3E 98 32 73 55 21 00 00 22 80 55 18 1E  .
      (>..2sU!..."..U..
      14C0: 3A 73 55 47 E6 0F FE 08 C2 19 2A 2A 80 55 78 87
      :sUG.....**..Ux.
      14D0: 38 0C 3E 18 CD 35 15 3E C3 DC 79 23 18 13 3E C3
      8.>..5.>..y#...>.
      14E0: CD 79 23 2A 69 55 2B 2B 22 80 55 21 73 55 CD 39
      .y#*iU++".U!sU.9
      14F0: 08 CD 1A 25 FE 3B C2 19 2A C3 1A 25 FE 3B C2 19
      ...%.;...*...%.;...
      +0 +1 +2 +3 +4 +5 +6 +7 +8 +9 +A +B +C +D +E +F  0123456789ABCDEF
      1500: 2A 3A E9 55 B7 28 12 3E C3 2A EA 55 CD 79 23 2A  *:.U.
      (>.*.U.y#*
      1510: 69 55 2B 2B 22 EA 55 18 05 3E C9 CD 5E 23 C3 1A
      iU++".U..>..^#..
      1520: 25 FE 3B C2 19 2A 3E C3 2A 63 55 11 03 00 19 CD
      %.;...*>.*cU.....
      1530: 79 23 C3 1A 25 E5 F5 ED 5B 69 55 13 13 B7 ED 52  y#...%...
      [iU....R
      1540: D1 5D 7D 87 9F BC E1 20 05 CD 6F 23 B7 C9 37 C9  .]}....
      ..o#...7.
      1550: DD E1 2A D7 55 E5 2A D9 55 E5 21 00 00 22 D9 55
      ..*.U.*.U.!..."..U
      1560: 2A 69 55 22 D7 55 DD E9 DD E1 2A D9 55 7C B5 C4
      *iU".U....*.U|..

```

One of the features of the Stellar language is that the basic data length is 8 bits. (This is the same as TL/1 for the 68 series.)

However, since there are functions available to access a 64KB memory range, it doesn't seem to be much of a problem. Also, the control structures, such as if, while, until, for, and loop statements, make it relatively easy to write programs.

Of particular note are the availability of inline statements, which allow machine code instructions and data to be inserted into the program, and the existence of statements that manipulate index registers IX and IY.

The book included file dump program fdump and Tower of Hanoi program hanoi as examples, so I used them as references to create a memory dump program called mdump.

```

1      /* memory dump */
2      prog  mdump();
3      cons   ngetchr := 1,                      /* get chr from console */
4      nputchr := 2;                            /* put chr to console */
5      var    d, i,
6      sttadr[4], endadr[4],
7      sadr[2], eadr[2],
8      ch;
9      data  msttadr: "sttadr= $",0,
10     mendadr: "endadr= $",0,
11     header: "      +0 +1 +2 +3 +4 +5 +6 +7 +8 +9 +A +B +C +D +E +F
12     0123456789ABCDEF",0;
13     {
14     /* input disp adrs area */
15     putstr(;.msttadr);
16     for i:=0 to 3 {
17     sttadr[i] := getchr();
18     }
19     putnl();
20     putstr(;.mendadr);
twenty  for i:=0 to 3 {
one     endadr[i] := getchr();
twenty  }
two     putnl(2);
twenty  sadr[0] := (sttadr[0]-$30)*$10+(sttadr[1]-$30);
three   sadr[1] := (sttadr[2]-$30)*$10+(sttadr[3]-$30);
twenty  eadr[0] := (endadr[0]-$30)*$10+(endadr[1]-$30);
four    eadr[1] := (endadr[2]-$30)*$10+(endadr[3]-$30);
twenty  /* disp memory data */
five    sadr[1] := sadr[1] & $f0;
26      /* start from adrlow=$x0 */
27      {
28      putstr(;.header);
29      putnl();
30      {
31      puthex(sadr[0]); puthex(sadr[1]);putchr(':');putchr(' '); /* disp
32      adrs */
33      for i:=0 to 15 {

```

```

34     puthex(memory[sadr[0], (sadr[1])+i]); putchar(' ');          /* disp by
35     hex */
36     }
37     putchar(' ');
38     for i:=0 to 15 {
39     d:=memory[sadr[0], (sadr[1])+i];
40     ch:= ?(d>=$20 & d<=$7e;d, '.');
41     putchar(ch);                                                  /* disp by
42     ascii */
43     }
44     putnl();
45     sadr[1] := sadr[1] + $10;
46     } until (((sadr[0] <> eadr[0]) & (sadr[1] = $00)) | ((sadr[0] =
47     eadr[0]) & (sadr[1] > eadr[1])));
48     putnl();
49     sadr[0] := sadr[0] + 1;
50     } until ((sadr[0] > eadr[0]));
51     }
52     /***** subroutine *****/
53     getch();
54     {
55     bdos(ngetchr);
56     }
57     putchar(x);
58     {
59     bdos(nputchr, x);
60     }
61     putnl(n);
62     cons
63     cr := $0d,
64     lf := $0a;
65     var
66     pn at ( _work );
67     {
68     if pn=0 then n:=1;
69     loop #,n {
70     bdos(nputchr, cr);
71     bdos(nputchr, lf);
72     }

```

```

73     }
74     putstr(;ix);
75     var x;
76     {
77     while x:=@[ix+] {
78     bdos(nputchr, x);
79     }
80     }
81     puthex(h);
82     {
83     puthex1(h>>4);
84     puthex1(h);
85     }
86     puthex1(h);
87     {
88     bdos(nputchr, ?((h:=h & $0f)<10 0005h="" _work="" a="" argn="1" at=""
89     bdos="" br="" c="" call="" cd="" d1="" dd="" de="" e1="" e5="" e=""
90     else="" f="" fd="" func="" h="" hl="" if="" inline="" ix="" iy=""
91     ld="" pop="" push="" then="" var="" x=""><!--10-->
92
93
94
95
96
97
98
99
100

```

mdump

As this example shows, you can directly call CP/M's BDOS, making it easy to create I/O-related programs. Therefore, I get the impression that it's a language well-suited for creating machine-specific tool software like GAME or TL/1. This concludes my report on creating a simple sample program using the CP/M version of the 8-bit compiler for the Z80 that was included in the book.

If anyone is interested in trying it out, please contact me by email and I can send you the executable files (stellar.com and convobj.com) as attachments in a reply email. (However, it will be quite difficult to use without reading the explanations of syntax and grammar in the book.)